

Lab # 1

Objectives

- To Familiarize with the Linux command line interpreter
- To be able to execute and interpret Linux basic commands

Pre-Lab Theory:

There are 2 ways to use the command

1. Absolute mode
2. Symbolic mode

1. Absolute(Numeric) Mode in Linux

In this mode, file permissions are not represented as characters but a three-digit octal number.

The table below gives numbers for all permissions types.

Number	Permission Type	Symbol
0	No Permission	—
1	Execute	x
2	Write	-w-
3	Execute + Write	-wx
4	Read	r—
5	Read + Execute	r-x
6	Read +Write	rw-
7	Read + Write +Execute	rxw

2. Symbolic Mode in Linux

The categories of people's access:

- a all users
- u the owner user
- g the owner group

- o others (neither u, nor g)

The format for permissions is:

chmod {a,u,g,o} {+,-} {r,w,x} files

The plus ("+") sign indicates give permission. The minus ("-") sign indicates remove permission.

Permission examples:

- chmod a+r files are readable by all
- chmod a-r files cancels the ability for all to read the file
- chmod a-rwx cancels all access for all
- chmod g+rw files give the group read and write permission
- chmod u+rwx files give the owner all permissions
- chmod og+rw files give the world and the group read and write permission

1. PATH:

a. Absolute Path

i. From root directory

/root/home/...

b. Relative Path

i. From current directory

./mydirectory

2. .(dot) and ..(dot dot)

a. .(dot) refers to current directory

b. ..(dot dot) refers to one level up directory

In-Lab Tasks

Linux Commands:

3. Present Working Directory:

\$pwd

Task 1: write the output

/home/muhammad-ahmad

4. List Files and Subdirectories:

- a. `$ls`
- b. `$ls -l`

Task: Explain all the columns in the output Also interpret the first character of the first column values Possible characters (-, d, l, p, s, b, c)

Command output:

```
-rw-r----- 1 muhammad-ahmad muhammad-ahmad 284 Sep  4 13:01 xyz.dat
```

Explanation:

first col : file-type , permissions
second col:
third col : owner of the machine (root user)
fourth col: group of the user
fifth col :size
sixth col : last modified
seven col : name of the file directory or other files

Interpretation of the first coloumn

interpretation of first character of first col :

```
-rw-r--r--    → Regular file  
drwxr-xr-x    → Directory  
lrwxrwxrwx    → Symbolic link  
prw-r--r--    → Named pipe  
srwxr-xr-x    → Socket  
brw-rw----    → Block device  
crw-rw----    → Character device
```

c. Display the Hidden files

`$ls -a`

Task 2: write the output

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ ls -a  
.  ..  del  xyz.dat
```

d. Metacharacters/wild card

- i. `$ls ch*.doc`

Task 3: First execute `ls` and then list some files through wild card and write output

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ ls  
del  xhin.dat  xyz.dat  
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ ls x*.dat  
xhin.dat  xyz.dat
```

5. Change Directory:

a. `$cd ../`

Change the directory to one level up

Task 4: execute 'pwd' first then 'cd ../' and write the output

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ pwd
/home/muhammad-ahmad/lab-tasks/lab-1
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ cd ../
muhammad-ahmad@latitude-7490:~/lab-tasks$ pwd
/home/muhammad-ahmad/lab-tasks
```

6. Creating Subdirectories:

`$mkdir <subdirectory path>`

Task 5: write the command to create subdirectory Lab1 in pwd

```
mkdir Lab1
```

`$mkdir -p <subdirectory path with parent directories>`

Parent directories will be created if not exist

Task 6: write the command to create directories with the following hierarchy

`/Linux_Commands/Lab_Tasks`

```
muhammad-ahmad@latitude-7490:~$ sudo mkdir -p /Linux_Commands/Lab_Tasks
-- we use sudo as / mean start from root and to access root we need admin access
```

7. Removing Subdirectories

`$rmdir`

Task 7: write the command to create a subdirectory Temp in the current directory and then write the command to remove it

```
create: muhammad-ahmad@latitude-7490:~$ mkdir delete
delete: muhammad-ahmad@latitude-7490:~$ rmdir delete
```

8. Copy Files

\$cp <source file path> <destination file path>

Task 8: create and edit a file temp.dat using gedit with some text and save the file in the Lab1 directory.

Write the command to copy temp.dat to subdirectory Lab_Tasks created in the previous task

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ cp temp.dat ../../lab-tasks/
```

9. Renaming the file

\$mv <file to rename> <renamed file>

Task 9: write a command to rename a file 'temp.dat' to 'myfile.dat' and execute 'ls' to verify it.

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ mv temp.dat myfile.dat
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ ls
myfile.dat
```

10. Removing/Deleting the file

\$rm <filename>

Task 10: write a command to remove the file temp.dat from the Lab_Tasks subdirectory

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ rm ../temp.dat
```

11. Display the Contents of the file on the screen

\$cat <file path>

Task 11: create a file using \$gedit xyz.dat type something, save the file, and exit

Write a command to display the xyz.dat file contents

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ cat xyz.dat
hello :)
```

12. Changing the permissions of the file

\$chmod <nnn> <filename>

\$chmod 777 xyz.txt

Task 12a: execute `ls -l` and write the permissions column value for `xyz.txt` file.

Command output:

```
-rw-rw-r-- 1 muhammad-ahmad muhammad-ahmad    0 Sep  5 00:44 xyz.txt
```

Explanation:

```
-rw-rw-r--  
- : regular file  
rw- : {owner/user} ==> read + write  
rw- : {group} ==> read + write  
r-- : {other} ==> read
```

Task 12b: write the command ***chmod 640 xyz.txt*** using symbolic mode

```
user  : group   : others  
6=110 : 4=100   : 0=000 | {format:rwX}  
u=rw   : g=r     : o=
```

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ chmod u=rw,g=r,o= xyz.dat
```

13. Display the word count of the file

`$wc <filename>`

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ wc xyz.dat  
 2  2 10 xyz.dat
```

Explanation:

```
{-l -w -c} : {line word bytes}
```

Task 13 write the command to display the word count of any file and
Interpret the output values

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ wc -w xyz.dat  
2 xyz.dat
```

Explanation:

```
file content:  
1- hello :)  
2-
```

```
hello -> 1 word  
:) -> 1 word  
Line 2 (empty line) -> 0 word
```

so we have total 2 words

- a. Display the PATH environment variable
\$echo \$PATH

Task 14: write the output

```
muhammad-ahmad@latitude-7490:~$ echo $PATH
/home/muhammad-ahmad/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/muhammad-ahmad/.local/bin:/home/muhammad-ahmad/.local/bin
```

Explanation:

\$PATH is an environment variable that tells the shell where to look for executable files when you run a command.

14.Redirection Operator (>)

Task15a: Execute \$ls -l > xyz.dat and explain the behavior

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ ls -l > xyz.dat
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ cat xyz.dat
total 4
drwxrwxr-x 2 muhammad-ahmad muhammad-ahmad 4096 Sep  4 11:00 del
-rw-r----- 1 muhammad-ahmad muhammad-ahmad    0 Sep  4 11:24 xyz.dat
```

Explanation:

ls -l → list file content in detail

> xyz.dat → store the previous output of ls -l in the file

Note:

it delete the existing content of the file and add new content which is the output of our command ls -l

Task 15b: Append some contents in the existing file 'xyz.dat'

Write the command string used to achieve the above task

Note:

for append the existing file we use >> instead of >

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ ls -l >> xyz.dat
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ cat xyz.dat
total 4
drwxrwxr-x 2 muhammad-ahmad muhammad-ahmad 4096 Sep  4 11:00 del
-rw-r----- 1 muhammad-ahmad muhammad-ahmad    0 Sep  4 11:24 xyz.dat
total 8
drwxrwxr-x 2 muhammad-ahmad muhammad-ahmad 4096 Sep  4 11:00 del
-rw-r----- 1 muhammad-ahmad muhammad-ahmad 142 Sep  4 11:24 xyz.dat
```

15.Top Command:

\$top

Task 16: Execute top command and interpret the output. Explain all the columns

line-1:{system-summary}

top - 11:52:26 up 2:57, 1 user, load average: 0.29, 0.47, 0.44

explanation:

- 11:52:26 -> Current system time
- up 2:57 -> System uptime (how long the system has been running -> here ~3 hours)
- 1 user -> Number of users logged in
- load average: 0.29, 0.47, 0.44 -> Average CPU load(on cores) over the last 1, 5, and 15 minutes

line-2:{tasks summary}

Tasks: 290 total, 1 running, 288 sleeping, 1 stopped, 0 zombie

explanation:

- 290 total -> total no of processes
- 1 running -> processes curently running
- 288 sleep -> processss curently at rest
- 1 stopped -> processes paused (ctrl+z)
- 0 zombie -> processes dead but not removed yet

line-3:{cpu usage}

%Cpu(s): 1.2 us, 0.8 sy, 0.0 ni, 98.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st

explanation:

- us (1.2%) -> User processes (apps you run)
- sy (0.8%) -> System (kernel tasks)
- ni (0.0%) -> Nice processes (with adjusted priority)
- id (98.0%)-> Idle time (CPU doing nothing → good sign here)
- wa (0.0%) -> Waiting for I/O (disk, network)
- hi (0.0%) -> Hardware interrupts
- si (0.0%) -> Software interrupts
- st (0.0%) -> Time stolen by hypervisor

line-4:{ram usage}

MiB Mem : 11839.7 total, 6233.3 free, 2800.0 used, 3509.9 buff/cache

explanation:

- 11839.7 total -> Total RAM (≈ 11.8 GB)
- 6233.3 free -> Currently unused
- 2800.0 used -> Actively used by processes
- 3509.9 buff/cache -> Used for buffers + cache (Linux uses free RAM as cache to speed up)

line-5:{swap usage/Virtual memory (swap is the dedicated portion 4gb on linux which act as ram if the ram is fully allocated)}

MiB Swap: 4096.0 total, 4096.0 free, 0.0 used. 9039.7 avail Mem

explanation:

- 4096.0 total -> Swap space (≈ 4 GB)
- 4096.0 free -> Unused swap
- 0.0 used -> Swap not used (good, means RAM is enough)
- 9039.7 avail Mem -> Available memory for new processes (includes free + reclaimable cache)

Columns detail:

PID : process id

USER: user associated to that process

PR : priority of the process

NI : nice value

VIRT: virtual mem size {Browsers apps reserve large virtual address spaces for performance, but they don't actually consume it in RAM}

RES : resident mem (actual ram)

SHR : shared memory (part of RES that can be shared with other processes)

S : process state (Running , Sleep , Zombie , Idle kernel thread)

CPU : cpu used by process

MEM : mem used by process

TIME+ COMMAND : {min:sec:milli-sec} running command

16. Listing the processes

\$ps

Task 17: write and interpret the output

```
muhammad-ahmad@latitude-7490:~/lab-tasks/lab-1$ ps
```

PID	TTY	TIME	CMD
9804	pts/1	00:00:00	bash
9968	pts/1	00:00:00	vim
13498	pts/1	00:00:00	ps

explanation:

ps only show the processes of this running terminal only, not the whole machine processes, for that purpose we have command ps -ef

17. \$gedit myfile.dat &

Task 18: Execute the above command and explain the behavior and importance of &

it open the editor for myfile.dat in detached mode

with & : detached mode / run in background

without & interactive mode / run in foreground killing the terminal also terminate the process running